Ingeniería de las Telecomunicaciones

# Guía del laboratorio

| | | | |
|---|---|---|---|
| **Laboratorio N°:** | 5 | **Semestre:** | 2026-2 |
| **Tema:** | Programación reactiva de redes SDN con ONOS | | |
| **Profesor:** | Christian Quispe | | |
| **Jefe(s) de práctica:** | Nilton Condori Q. (nilton.condori@pucp.edu.pe) | | |

**Indicaciones generales:**

- **Tiempo estimado:** 2 horas.
- **Puntaje total:** 5 ptos.
- **Entrega:** al finalizar la sesión, en un único archivo PDF subido a PAIDEIA, sección “LABORATORIO 5 - EXPERIENCIA DE LABORATORIO”, con el nombre TEL354_RF_LAB5_<CODIGO_PUCP>.pdf.
- El desarrollo es personal.
- **Requisito:** haber completado el informe previo, en especial el último inciso de la Actividad 5 (slice_reset.yml).
- Todos los comandos se entregan listos para copiar. El puntaje corresponde a la interpretación de los resultados: en cada recuadro “# Responder aquí.” adjunte las capturas solicitadas y responda las preguntas.
- **Credenciales:** usuario ubuntu / contraseña ubuntu en todas las VM (también para sudo); ONOS: usuario onos / contraseña rocks.

**Resultados de aprendizajes**

Al finalizar el laboratorio, serás capaz de:

1. Desplegar el controlador ONOS en un entorno contenerizado y programar la red en modo reactivo, seleccionando y configurando sus aplicaciones.

2. Evaluar el comportamiento del plano de control en los enfoques reactivo y proactivo a partir de los mensajes PACKET_IN, PACKET_OUT y FLOW_MOD, relacionándolo con la latencia y la escalabilidad de la red.

**Objetivos del laboratorio**

1. Desplegar ONOS desde una imagen preconfigurada y realizar el bootstrap de la red con Ansible.
2. Analizar el intercambio PACKET_IN / FLOW_MOD / PACKET_OUT de una aplicación reactiva (fwd) y modificar su comportamiento.
3. Implementar un control de acceso por roles (RBAC) reactivo, en el que cada PACKET_IN se resuelve con un intent o una regla DROP según una política.
4. Comparar los enfoques reactivo y proactivo en latencia, carga del controlador, reglas instaladas y reenvío a line-rate.

**Conceptos fundamentales:**

**Procesamiento de paquetes en ONOS.** Una aplicación reactiva solicita al PacketService los paquetes que le interesan (por ejemplo, IPv4); ONOS instala en cada switch una regla que envía ese tráfico al controlador. Cada PACKET_IN se entrega a los PacketProcessor registrados, en orden de prioridad. El procesador decide qué hacer con el paquete: instalar una regla (FLOW_MOD), reenviarlo (PACKET_OUT), o ambas cosas.

**Aplicación fwd (Reactive Forwarding).** Por cada PACKET_IN de IPv4 calcula el siguiente salto hacia el host destino, instala en ese switch una regla temporal y reenvía el paquete con PACKET_OUT. ONOS borra la regla (FLOW_MOD DELETE) cuando lleva flowTimeout segundos sin tráfico. Sus propiedades se modifican en tiempo de ejecución con cfg set org.onosproject.fwd.ReactiveForwarding <propiedad> <valor>; por ejemplo, flowTimeout (segundos sin tráfico tras los que ONOS borra una regla de fwd), matchIpv4Address (incluir las direcciones IP en el match) y packetOutOnly (no instalar reglas: reenviar cada paquete por el controlador).

**Control de acceso reactivo (RBAC).** La decisión también puede tomarla un programa externo. La app packet-bridge (Java, dentro de ONOS) solicita los paquetes IPv4 y reenvía cada PACKET_IN por HTTP a una aplicación en Python. Esta consulta una política por roles (policy.yaml) y, si el tráfico está permitido, crea un HostToHostIntent por la API REST: ONOS compila el intent en reglas en todos los switches del camino y,

desde el segundo paquete, el tráfico se reenvía a line-rate. Si está denegado, instala una regla DROP temporal en el switch de ingreso.

**Reenvío a line-rate.** Mientras un paquete coincide con una regla de la tabla, el switch lo reenvía en su datapath (en un switch físico, el ASIC) sin intervención del controlador. Cada paquete que genera un PACKET_IN, en cambio, recorre el canal de control y el software del controlador.

**API REST de ONOS y módulos reutilizables.** Así como Floodlight documenta su API REST (laboratorio 4), ONOS publica la suya en http://<IP>:8181/onos/v1/ con documentación interactiva en /onos/v1/docs/; la documentación general de ONOS y µONOS está en https://docs.onosproject.org. El Anexo B resume los recursos que puede reutilizar en su proyecto y su equivalente en Floodlight; en particular, POST /flows/{deviceId} cumple el papel del Static Entry Pusher y GET /paths/{src}/{dst} el de /wm/topology/route.

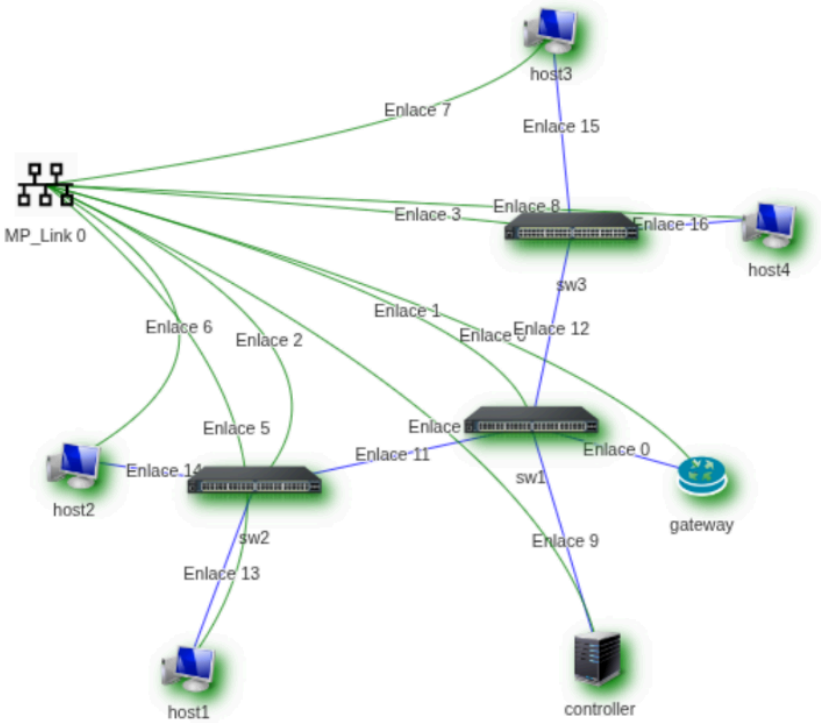
**Protocolos southbound.** Este laboratorio trabaja con OpenFlow, pero ONOS soporta otros protocolos mediante proveedores y drivers que se activan como apps: NETCONF, gNMI con modelos OpenConfig y P4Runtime. Para la configuración de equipos con gNMI/OpenConfig existe además el subsistema onos-config del proyecto µONOS (https://docs.onosproject.org/onos-api/docs/onos/config/admin/), con una API gRPC propia.

**Materiales y equipos a usar:**

- Slice desplegado en el DataCore, en el estado en que lo dejó el informe previo.
- Laptop con cliente SSH y navegador web.
- Repositorio del laboratorio: https://github.com/NohaFeynman/TEL354_LAB5_2026_2
- Documentación oficial de ONOS (sitio actual de la Open Networking Foundation, que reemplaza a la antigua wiki): https://docs.onosproject.org
- Documentación de la API REST de ONOS 2.7: interfaz Swagger del propio controlador (http://localhost:8181/onos/v1/docs/) y copia comunitaria de la antigua wiki de ONOS: https://andrea-campanella.github.io/onos-classic-wiki/onos/guides/developer-guide/appendix-b-rest-api/
- ONF, OpenFlow Switch Specification 1.3.5:

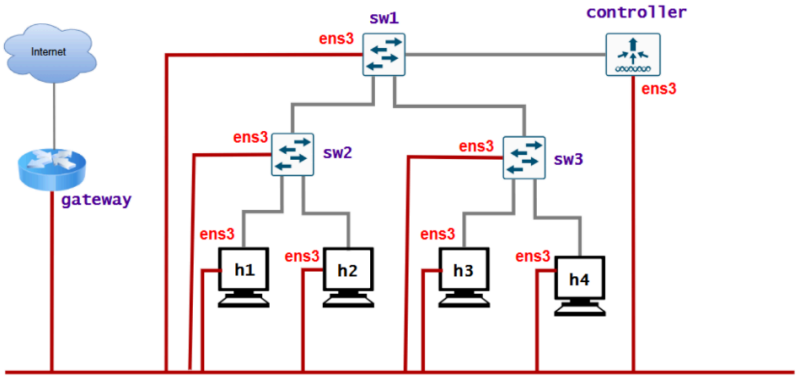
https://opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.3.5.pdf

**Topología del curso:**




A continuación se muestra la topología del curso, compuesta por dos tipos de redes:

- Red roja (ACCESS Network, 192.168.0.0/24): red de gestión. Transporta el SSH de Ansible y el canal OpenFlow entre los switches y ONOS (control fuera de banda).
- Red gris (SDN Network): plano de datos que conecta los switches OVS entre sí y con los hosts (10.0.0.0/24).

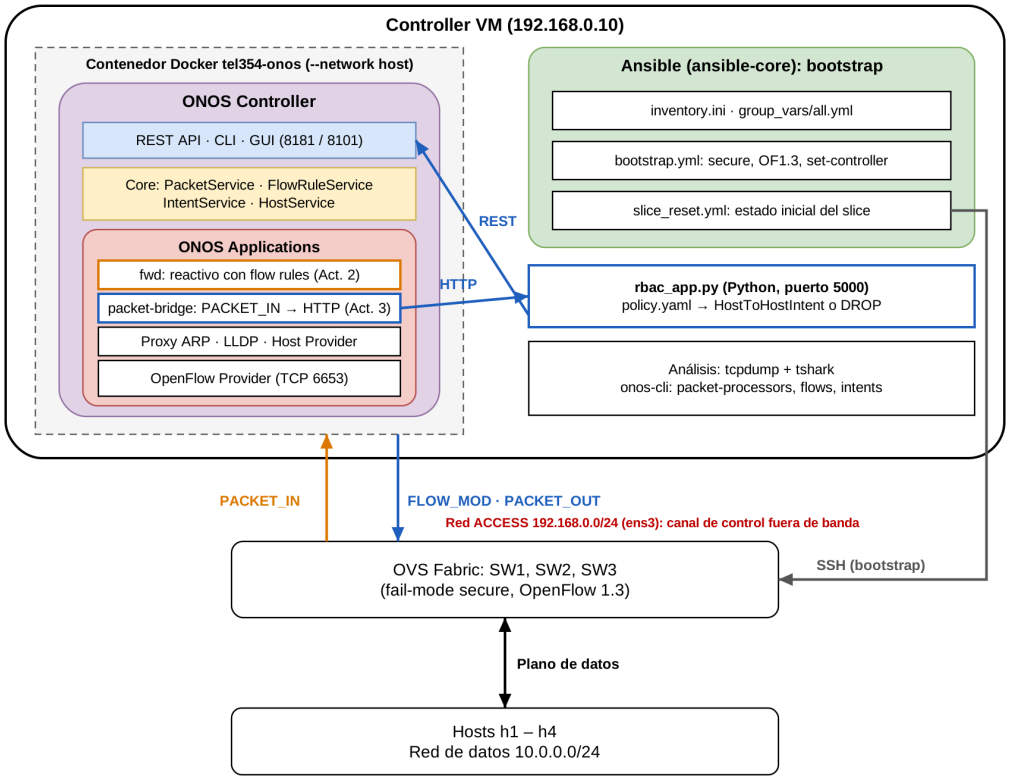



| Virtual Machine | PORT |
|---|---|
| Gateway | 22 |

| Virtual Machine | PORT |
|---|---|
| CONTROLLER | 5800 |
| SW1 | 5801 |
| SW2 | 5802 |
| SW3 | 5803 |
| H1 | 5811 |
| H2 | 5812 |
| H3 | 5813 |
| H4 | 5814 |

**Arquitectura de la experiencia:**

La figura muestra los componentes que se ejecutan en la VM controller. Ansible solo realiza el bootstrap de los switches por SSH; las reglas de tráfico las decide ONOS a partir de los PACKET_IN que envían los switches por el canal OpenFlow.




Elaboración propia a partir de la documentación de ONOS y de la especificación OpenFlow 1.3.5 (ONF).

*Figura 1. Arquitectura de la experiencia (elaboración propia).*

**Experiencia:**

En esta sección se brindan las indicaciones de la parte práctica. Cada actividad tiene asignado un puntaje y en cada recuadro "# Responder aquí." debe colocar su respuesta, incluyendo las capturas solicitadas.

**ACTIVIDAD 0: PREPARACIÓN (0 ptos)**

1. En el controller, verificar el estado que dejó el informe previo y actualizar el repositorio:

```
ubuntu@controller:~$ sudo docker ps -a; sudo docker images
ubuntu@controller:~$ cd ~/TEL354_LAB5_2026_2 && git pull
ubuntu@controller:~$ cd ansible && ansible switches -m ping
ubuntu@controller:~$ python3 -c "import yaml; print('PyYAML OK')"
ubuntu@controller:~$ sudo DEBIAN_FRONTEND=noninteractive \
  apt install -y tshark
```

**Verificación:** docker ps -a no debe listar contenedores, docker images debe mostrar onosproject/onos:2.7.0 y ansible switches -m ping debe responder pong en los tres switches. Si algo falta, complete primero las Actividades 2 y 3 del informe previo.

## ACTIVIDAD 1: ONOS CONTENERIZADO Y BOOTSTRAP DE LA RED (1.5 ptos)

1. **(0.5 ptos)** Imagen del laboratorio. Descargue e inspeccione la imagen preparada para la experiencia, derivada de la oficial:

```
ubuntu@controller:~$ cat ~/TEL354_LAB5_2026_2/docker/Dockerfile
ubuntu@controller:~$ IMG=ghcr.io/nohafeynman/tel354-onos:2026-2
ubuntu@controller:~$ sudo docker pull $IMG
ubuntu@controller:~$ sudo docker history $IMG | head -6
```

   Adjunte las salidas. ¿Por qué la descarga fue casi instantánea si la imagen ocupa unos 836 MB? Explique qué configura cada instrucción del Dockerfile (ONOS_APPS, JAVA_OPTS y HEALTHCHECK) y por qué el chequeo de salud verifica el puerto 6653.

```
# Responder aquí.
root@controller:~# sudo docker pull $IMG
2026-2: Pulling from nohafeynman/tel354-onos
7595c8c21622: Already exists
d13af8ca898f: Already exists
70799171ddba: Already exists
b6c12202c5ef: Already exists
a3caae5bc1ad: Already exists
ce1773b9f958: Already exists
1f74a437590f: Already exists
Digest: sha256:520551edc2fa6309704e447907ca80a52cec3407ab21fbc304dbaf884c13f642
Status: Downloaded newer image for ghcr.io/nohafeynman/tel354-onos:2026-2
ghcr.io/nohafeynman/tel354-onos:2026-2
root@controller:~# sudo docker history $IMG | head -6
IMAGE          CREATED       CREATED BY                                      SIZE      COMMENT
42dccfa8f8bc   4 years ago   HEALTHCHECK {Test:[CMD-SHELL bash -c 'exec 3…   0B        buildkit.dockerfile.v0
<missing>      4 years ago   EXPOSE [6653/tcp 8101/tcp 8181/tcp]             0B        buildkit.dockerfile.v0
<missing>      4 years ago   ENV JAVA_OPTS=-Xms512m -Xmx1g                   0B        buildkit.dockerfile.v0
<missing>      4 years ago   ENV ONOS_APPS=gui2,drivers,openflow,proxyarp    0B        buildkit.dockerfile.v0
<missing>      4 years ago   LABEL org.opencontainers.image.title=tel354-…   0B        buildkit.dockerfile.v0
root@controller:~# 4
```

Fue instantánea porque la imagen Docker ya se encontraba previamente almacenada en la caché local del demonio de Docker.

Por otro lado, ONOS_APPS configura la lista de aplicaciones que deben activarse como es el caso del soporte de GUI2, los drivers, controladores OpenFlow, el proxy ARP; mientras que JAVA_OPTS configura Xms521m para la memoria asignada y -Xmx1g para la memoria máxima para la máquina virtual de Java. Finalmente, HEALTHCHECK configura una prueba periódica para verificar el estado de salud interna del contenedor.

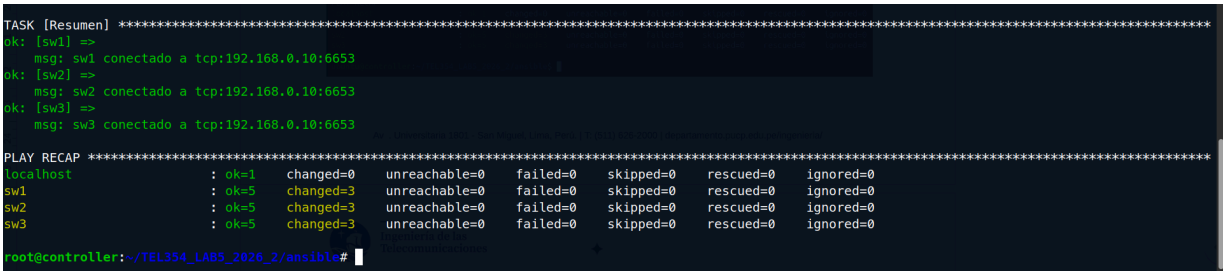
Respecto al estado de salud, el puerto TCP 6653 es el puerto estándar utilizado por el protocolo OpenFlow para el canal de control southbound, por lo que, verificar su apertura garantiza que el subsistema OpenFlow de ONOS está inicializado, escuchando y listo para aceptar las conexiones de los switches OVS.

2. **(0.5 ptos)** Despliegue y bootstrap. Levante ONOS, espere a que esté healthy y ejecute el bootstrap de los switches (pide BECOME password: ubuntu):

```
ubuntu@controller:~$ sudo docker run -d --name onos --network host \
  --restart unless-stopped $IMG
ubuntu@controller:~$ until sudo docker ps | grep -q '(healthy)'; do
  sleep 5; done; sudo docker ps
ubuntu@controller:~$ ansible-playbook bootstrap.yml
```

Adjunte el estado healthy y el PLAY RECAP. ¿Qué hizo Ansible en los switches y qué dejó en manos de ONOS?

```
# Responder aquí.
```




```
Ansible realizó el bootstrap fuera de banda conectándose por SSH a cada switch OVS para
configurar la versión del protocolo OpenFlow en OpenFlow13, establecer el modo de falla a
secure y apuntar el canal de control hacia la IP de ONOS.

Por otro lado, ONOS asumió el control total sobre el plano de datos y adicionalmente se
encargará de gestionar el descubrimiento dinámico de la topología física, mantener la base
de datos de ubicación de hosts y decidir o programar el reenvío de tráfico e instalación de
reglas de flujo mediante el canal OpenFlow.
```

3. **(0.5 ptos)** CLI. El contenedor genera una clave de host nueva, por eso primero se borra la anterior de known_hosts:

```
ubuntu@controller:~$ ssh-keygen -R "[localhost]:8101"
ubuntu@controller:~$ alias onos-cli='ssh -p 8101 \
  -o StrictHostKeyChecking=no -o HostKeyAlgorithms=+ssh-rsa \
  -o PubkeyAcceptedAlgorithms=+ssh-rsa onos@localhost'
ubuntu@controller:~$ onos-cli
onos@root > apps -a -s
onos@root > devices
onos@root > packet-processors
```

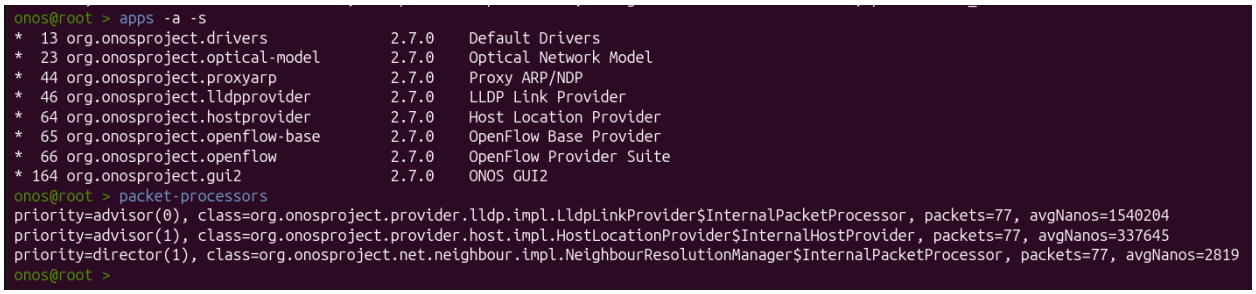


*Figura 2. Procesadores de paquetes de ONOS.*

Adjunte las salidas. ¿Qué aplicaciones registraron un PacketProcessor y para qué procesan paquetes? ¿Por qué ninguna de ellas reenvía todavía el tráfico IPv4 entre hosts?

**Pista:** NeighbourResolutionManager es el servicio que usa la app proxyarp para responder los ARP. Los procesadores advisor solo observan los paquetes; los director pueden decidir qué hacer con ellos.

```
# Responder aquí.
```

```
onos@root > apps -a -s
*  13 org.onosproject.drivers             2.7.0    Default Drivers
*  23 org.onosproject.optical-model       2.7.0    Optical Network Model
*  44 org.onosproject.proxyarp            2.7.0    Proxy ARP/NDP
*  46 org.onosproject.lldpprovider        2.7.0    LLDP Link Provider
*  64 org.onosproject.hostprovider        2.7.0    Host Location Provider
*  65 org.onosproject.openflow-base       2.7.0    OpenFlow Base Provider
*  66 org.onosproject.openflow            2.7.0    OpenFlow Provider Suite
* 165 org.onosproject.gui2                2.7.0    ONOS GUI2
onos@root > devices
onos@root > packet-processors
priority=advisor(0), class=org.onosproject.provider.lldp.impl.LldpLinkProvider$InternalPacketProcessor, packets=0, avgNanos=0
priority=advisor(1), class=org.onosproject.provider.host.impl.HostLocationProvider$InternalHostProvider, packets=0, avgNanos=0
priority=director(1), class=org.onosproject.net.neighbour.impl.NeighbourResolutionManager$InternalPacketProcessor, packets=0, av
onos@root >
```

```
Aplicaciones que registraron un PacketProcessor y sus funciones:
```

- `LldpLinkProvider$InternalPacketProcessor: procesa tramas de control LLDP y BDDP para descubrir los enlaces activos inter-switch y construir la topología de la red.`
- `HostLocationProvider$InternalHostProvider: procesa paquetes recibidos (como ARP/IPv4) para aprender y actualizar la ubicación física (como el switch ID y puerto) de los hosts finales.`
- `NeighbourResolutionManager$InternalPacketProcessor: intercepta solicitudes ARP/NDP para responderlas centralizadamente o dirigirlas directamente al destino conocido sin requerir inundación masiva.`

```
Ninguna de ellas reenvía tráfico IPv4 de los hosts porque ninguna de estas aplicaciones está diseñada para cumplir funciones de enrutamiento o conmutación L2/L3 de tráfico de datos. Solamente ejecutan tareas de descubrimiento de topología, localización de hosts y resolución de direcciones y no existe entonces una aplicación de reenvío activa como fwd que calcule rutas para paquetes IPv4 e instale reglas de flujo en los switches.
```

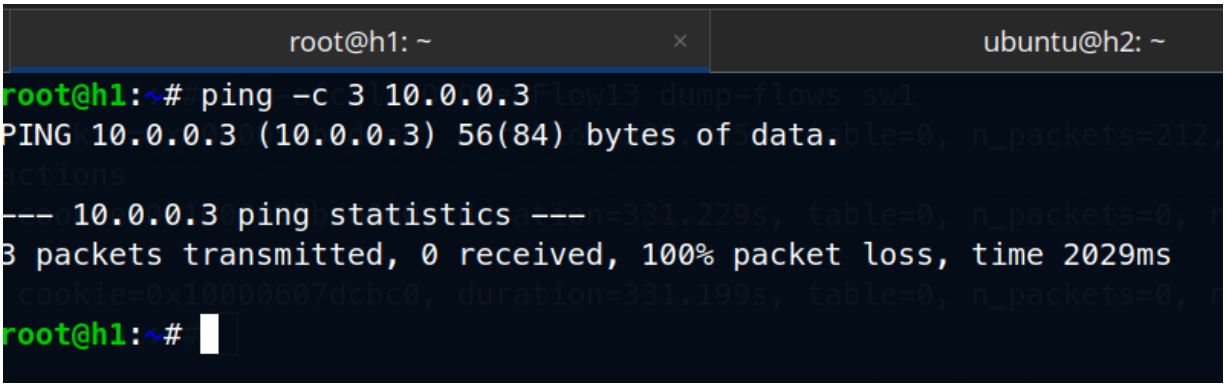
## ACTIVIDAD 2: ENFOQUE REACTIVO CON LA APLICACIÓN fwd (1.5 ptos)

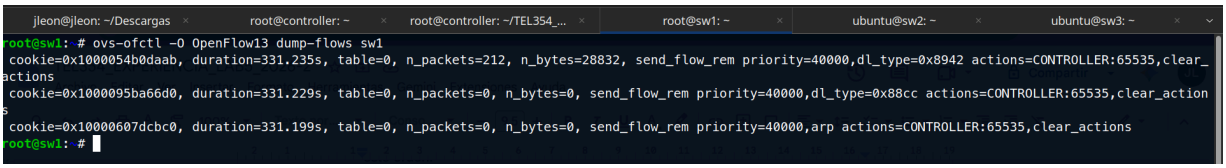
1. **(0.25 ptos)** Antes de fwd. Pruebe la conectividad y revise las reglas de sw1:

```
ubuntu@h1:~$ ping -c 2 10.0.0.3
ubuntu@sw1:~$ sudo ovs-ofctl -O OpenFlow13 dump-flows sw1
```

¿Por qué falla el ping si los paquetes ICMP no llegan al controlador? Explique cada una de las reglas de sw1 (match y acción) y qué hace un switch OpenFlow 1.3 con un paquete que no coincide con ninguna regla.

```
# Responder aquí.
```




- Falla la prueba de conectividad debido porque los paquetes ICMP no hacen match con ninguna de las tres reglas instaladas en el switch; al no haber una regla que capture el tráfico IPv4 ni una regla por defecto que envíe este tráfico al controlador, el switch descarta los paquetes en el datapath.

- Explicación de las reglas:

a. priority=40000, dl_type=0x88cc actions=CONTROLLER:65535: captura tramas LLDP y las envía al controlador para el descubrimiento de enlaces inter-switch.

b. priority=40000, dl_type=0x8942 actions=CONTROLLER:65535: captura tramas BDDP y las envía al controlador para el descubrimiento de enlaces cuando se usan switches no compatibles con OpenFlow estándar.

c. priority=40000, dl_type=0x0806 actions=CONTROLLER:65535: captura paquetes ARP y los reenvía al controlador para su resolución por parte de proxyarp.

- En la especificación OpenFlow 1.3, si es que la tabla de flujo no contiene una regla de "table-miss" explícita, el comportamiento predeterminado del switch es descartar el paquete.

2. **(0.5 ptos)** Activar fwd y capturar el intercambio reactivo. Ejecute los pasos en este orden:

**Paso 1 (CLI de ONOS):** active fwd y revise la regla nueva de sw1:

```
onos@root > app activate org.onosproject.fwd
ubuntu@sw1:~$ sudo ovs-ofctl -O OpenFlow13 dump-flows sw1
```

**Paso 2 (controller, terminal 2):** inicie una captura de 30 s:

```
ubuntu@controller:~$ sudo timeout 30 tcpdump -U -i ens3 -nn \
  tcp port 6653 -w ~/lab5_fwd.pcap
```

**Paso 3 (h1), sin esperar a que termine la captura:** envíe 5 pings y observe el RTT de cada uno:

```
ubuntu@h1:~$ ping -c 5 10.0.0.3
```

**Paso 4 (controller, terminal 2):** cuando tcpdump muestre su resumen, resuma los mensajes OpenFlow del ping con el script del laboratorio (tools/of_resumen.py):

```
ubuntu@controller:~$ python3 ~/TEL354_LAB5_2026_2/tools/of_resumen.py
~/lab5_fwd.pcap
```

**Nota:** el script muestra, en orden, cada PACKET_IN y PACKET_OUT que transporta el ping y cada FLOW_MOD, con el switch que lo envió o recibió y sus campos principales; al final imprime los totales, separando los FLOW_MOD que instalan reglas (ADD) de los que las borran (DELETE). Omite las sondas LLDP y BDDP que ONOS intercambia con los switches para descubrir la topología. Si las líneas se cortan, maximice la terminal.

Adjunte la salida. Responda: (a) ¿Qué regla nueva instaló fwd al activarse y para qué? (b) ¿Cuántos PACKET_IN, PACKET_OUT y FLOW_MOD ADD generó el ping (línea Total)? Justifique el número con los switches del camino y los dos sentidos del tráfico, y compárelo con su hipótesis de la Actividad 5 del informe previo. (c) En la salida del script, tome la primera línea de cada tipo e identifique: en el PACKET_IN, el motivo (reason), el puerto de entrada y el paquete encapsulado; en el FLOW_MOD, el match, la acción, la prioridad y el idle_timeout; y en el PACKET_OUT, la acción de salida.

```
# Responder aquí.
```

```
root@controller:~# sudo timeout 30 tcpdump -U -i ens3 -nn \
  tcp port 6653 -w ~/lab5_fwd.pcap
tcpdump: listening on ens3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
414 packets captured
414 packets received by filter
0 packets dropped by kernel
```

```
root@controller: # python3 ~/TEL354_LAB5_2026_2/tools/of_resumen.py ~/lab5_fwd.pcap
  t (s)  switch mensaje     detalle
  0.000  sw2    PACKET_IN   h1 -> h3 echo request seq=1 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
  0.013  sw2    PACKET_OUT  h1 -> h3 echo request seq=1 | acción: output:1
  0.016  sw1    PACKET_IN   h1 -> h3 echo request seq=1 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
  0.017  sw1    PACKET_OUT  h1 -> h3 echo request seq=1 | acción: output:3
  0.020  sw3    PACKET_IN   h1 -> h3 echo request seq=1 | reason=ACTION (regla con salida CONTROLLER) | in_port=3
  0.021  sw3    PACKET_OUT  h1 -> h3 echo request seq=1 | acción: output:2
  0.023  sw3    PACKET_IN   h3 -> h1 echo reply seq=1 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
  0.027  sw3    PACKET_OUT  h3 -> h1 echo reply seq=1 | acción: output:3
  0.028  sw1    PACKET_IN   h3 -> h1 echo reply seq=1 | reason=ACTION (regla con salida CONTROLLER) | in_port=3
  0.030  sw1    PACKET_OUT  h3 -> h1 echo reply seq=1 | acción: output:2
  0.032  sw2    PACKET_IN   h3 -> h1 echo reply seq=1 | reason=ACTION (regla con salida CONTROLLER) | in_port=1
  0.034  sw2    PACKET_OUT  h3 -> h1 echo reply seq=1 | acción: output:2
  0.044  sw2    FLOW_MOD    ADD | match: in_port=1 eth_dst=fa:16:3e:88:05:79 eth_src=fa:16:3e:05:4c:50 | acción: output:2 | prioridad=10 idle_timeout=0 hard_t
imeout=0
  0.045  sw3    FLOW_MOD    ADD | match: in_port=3 eth_dst=fa:16:3e:05:4c:50 eth_src=fa:16:3e:88:05:79 | acción: output:2 | prioridad=10 idle_timeout=0 hard_t
imeout=0
  0.045  sw1    FLOW_MOD    ADD | match: in_port=2 eth_dst=fa:16:3e:05:4c:50 eth_src=fa:16:3e:88:05:79 | acción: output:3 | prioridad=10 idle_timeout=0 hard_t
imeout=0
  0.045  sw1    FLOW_MOD    ADD | match: in_port=3 eth_dst=fa:16:3e:88:05:79 eth_src=fa:16:3e:05:4c:50 | acción: output:2 | prioridad=10 idle_timeout=0 hard_t
imeout=0
  0.045  sw3    FLOW_MOD    ADD | match: in_port=2 eth_dst=fa:16:3e:88:05:79 eth_src=fa:16:3e:05:4c:50 | acción: output:3 | prioridad=10 idle_timeout=0 hard_t
imeout=0
  0.046  sw2    FLOW_MOD    ADD | match: in_port=2 eth_dst=fa:16:3e:05:4c:50 eth_src=fa:16:3e:88:05:79 | acción: output:1 | prioridad=10 idle_timeout=0 hard_t
imeout=0
 18.508  sw3    FLOW_MOD    DELETE_STRICT | match: in_port=3 eth_dst=fa:16:3e:05:4c:50 eth_src=fa:16:3e:88:05:79 | acción: ninguna (drop) | prioridad=10 idle_
timeout=0 hard_timeout=0
 18.508  sw3    FLOW_MOD    DELETE_STRICT | match: in_port=2 eth_dst=fa:16:3e:88:05:79 eth_src=fa:16:3e:05:4c:50 | acción: ninguna (drop) | prioridad=10 idle_
timeout=0 hard_timeout=0

Total (mensajes del ping y FLOW_MOD): PACKET_IN=6, PACKET_OUT=6, FLOW_MOD ADD=6, FLOW_MOD DELETE=2
root@controller: #
```

```
root@h1:~# ping -c 5 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=37.9 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=2.21 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=1.78 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=1.70 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=1.66 ms

--- 10.0.0.3 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4007ms
rtt min/avg/max/mdev = 1.657/9.041/37.862/14.411 ms
root@h1:~#
```

```
root@sw1:~# ovs-ofctl -O OpenFlow13 dump-flows sw1
 cookie=0x1000054b0daab, duration=167725.447s, table=0, n_packets=108210, n_bytes=14716560, send_flow_rem priority=40000,dl_type=0x8942 actions=CONTROLLER:655
35,clear_actions
 cookie=0x1000095ba66d0, duration=167725.441s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=40000,dl_type=0x88cc actions=CONTROLLER:65535,clear_act
ions
 cookie=0x10000607dcbc0, duration=167725.411s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=40000,arp actions=CONTROLLER:65535,clear_actions
 cookie=0x100003b5d6ece, duration=167313.046s, table=0, n_packets=2, n_bytes=196, send_flow_rem priority=5,ip actions=CONTROLLER:65535,clear_actions
```

```
root@sw2:~# ovs-ofctl -O OpenFlow13 dump-flows sw2
 cookie=0x10000f6460ac7, duration=167764.077s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=40000,dl_type=0x88cc actions=CONTROLLER:65535,clear_act
ions
 cookie=0x1000012214b72, duration=167764.077s, table=0, n_packets=54118, n_bytes=7360048, send_flow_rem priority=40000,dl_type=0x8942 actions=CONTROLLER:65535
,clear_actions
 cookie=0x1000016815f2b, duration=167764.027s, table=0, n_packets=2, n_bytes=120, send_flow_rem priority=40000,arp actions=CONTROLLER:65535,clear_actions
 cookie=0x10000f338aa69, duration=167351.678s, table=0, n_packets=2, n_bytes=196, send_flow_rem priority=5,ip actions=CONTROLLER:65535,clear_actions
```

```
root@sw3:~# ovs-ofctl -O OpenFlow13 dump-flows sw3
 cookie=0x10000ab970cfe, duration=167796.731s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=40000,dl_type=0x88cc actions=CONTROLLER:65535,clear_act
ions
 cookie=0x10000fa290855, duration=167796.731s, table=0, n_packets=54128, n_bytes=7361408, send_flow_rem priority=40000,dl_type=0x8942 actions=CONTROLLER:65535
,clear_actions
 cookie=0x100005128531a, duration=167796.683s, table=0, n_packets=2, n_bytes=120, send_flow_rem priority=40000,arp actions=CONTROLLER:65535,clear_actions
 cookie=0x100007dc9c82d, duration=167384.332s, table=0, n_packets=2, n_bytes=196, send_flow_rem priority=5,ip actions=CONTROLLER:65535,clear_actions
```

a. El forwarding instaló la regla priority=5, ip actions=CONTROLLER:65535 que sirve para capturar cualquier paquete IPv4 que no coincida con reglas específicas preexistentes y enviarlo a ONOS como un PACKET_IN para que fwd calcule la ruta e instale los flujos.

b. PACKET_IN, PACKET_OUT, FLOW_MOD totales:

PACKET_IN = 2:
El primer paquete Echo Request (h1 -> h3) genera 1 PACKET_IN en el switch de ingreso sw2.
El primer paquete Echo Reply (h3 -> h1) genera 1 PACKET_IN en el switch de ingreso sw3.

PACKET_OUT = 2:
ONOS envía un PACKET_OUT para inyectar el Echo Request en sw3 hacia h3, y otro PACKET_OUT para inyectar el Echo Reply en sw2 hacia h1.

FLOW_MOD ADD = 6: Se instalan reglas de flujo para el sentido ida (3 switches: sw2, sw1, sw3) y 3 para el sentido vuelta (3 switches: sw3, sw1, sw2). En total, 3 x 2 = 6 mensajes FLOW_MOD ADD.Los 4 pings restantes se conmutan por completo en el datapath a line-rate, sin interactuar con el controlador.

c. Análisis:

- 0.000 sw2 PACKET_IN h1 -> h3 echo request seq=1 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
- 0.013 sw2 PACKET_OUT h1 -> h3 echo request seq=1 | acción: output:1
- 0.044 sw2 FLOW_MOD ADD | match: in_port=1 eth_dst=fa:16:3e:88:05:79 eth_src=fa:16:3e:05:4c:50 | acción: output:2 | prioridad=10 idle_timeout=0 hard_timeout=0

3. **(0.25 ptos)** Expiración de las reglas. Revise las reglas de fwd, espere a que expiren y repita el ping:

```
ubuntu@sw1:~$ sudo ovs-ofctl -O OpenFlow13 dump-flows sw1
ubuntu@sw1:~$ sleep 20; sudo ovs-ofctl -O OpenFlow13 dump-flows sw1
ubuntu@h1:~$ ping -c 3 10.0.0.3
```

¿Por qué el primer ping tiene un RTT mayor que los siguientes? ¿Qué ocurrió con las reglas después de 20 s y qué implica para el primer paquete del nuevo ping? En la salida del script del inciso 2, ¿qué valor tiene el idle_timeout de los FLOW_MOD ADD y qué son los FLOW_MOD DELETE_STRICT del final? ¿Quién borra las reglas de fwd: el switch o ONOS?

```
# Responder aquí.
```

- El primer paquete sufre la latencia del plano de control, es decir, el tiempo de encapsulamiento en PACKET_IN, procesamiento en ONOS para calcular la ruta e

instalación de reglas mediante FLOW_MOD. Los siguientes paquetes encuentran las reglas ya instaladas en la tabla TCAM del switch y se conmutan a velocidad de línea.

- Las reglas expiraron al transcurrir el idle_timeout de 10 segundos sin tráfico. El switch eliminó las reglas del datapath. Por ello, el nuevo ping vuelve a generar un PACKET_IN, experimentando nuevamente un RTT elevado en su primer paquete.
- El idle_timeout es de 10 segundos.

Por otro lado, los FLOW_MOD DELETE_STRICT son mensajes enviados para la eliminación explícita de flujos específicos cuando la aplicación se reconfigura o se limpia. Finalmente, el switch borra las reglas de forma autónoma al expirar el temporizador de inactividad (idle_timeout), aunque ONOS también puede enviar órdenes de borrado explícitas.

4. **(0.5 ptos)** Modificar el comportamiento de fwd. Cambie sus propiedades desde la CLI de ONOS en este orden:

**Paso 1 (CLI de ONOS, h1 y sw1):** revise las propiedades, aumente el timeout, incluya las IP en el match y repita el ping:

```
onos@root > cfg get org.onosproject.fwd.ReactiveForwarding
onos@root > cfg set org.onosproject.fwd.ReactiveForwarding flowTimeout 60
onos@root > cfg set org.onosproject.fwd.ReactiveForwarding matchIpv4Address true
ubuntu@h1:~$ ping -c 3 10.0.0.3
ubuntu@sw1:~$ sudo ovs-ofctl -O OpenFlow13 dump-flows sw1
```

**Nota:** cada comando cfg set va en una sola línea.

**Paso 2 (CLI de ONOS y terminal 1):** active el modo packetOutOnly y borre las reglas que fwd ya había instalado (si no, el ping las seguiría usando durante 60 s):

```
onos@root > cfg set org.onosproject.fwd.ReactiveForwarding packetOutOnly true
ubuntu@controller:~$ curl -s -u onos:rocks -X DELETE -o /dev/null \
  -w "HTTP %{http_code}\n" \
  http://localhost:8181/onos/v1/flows/application/org.onosproject.fwd
```

**Paso 3 (terminal 2, luego h1 sin esperar):** repita la captura de 30 s del inciso 2 con el nombre lab5_pkto.pcap y, mientras corre, envíe 5 pings:

```
ubuntu@controller:~$ sudo timeout 30 tcpdump -U -i ens3 -nn \
  tcp port 6653 -w ~/lab5_pkto.pcap
ubuntu@h1:~$ ping -c 5 10.0.0.3
```

**Paso 4 (terminal 2 y sw1):** cuando tcpdump muestre su resumen, obtenga los totales de la captura con el script y revise las reglas de sw1:

```
ubuntu@controller:~$ python3 ~/TEL354_LAB5_2026_2/tools/of_resumen.py
~/lab5_pkto.pcap
ubuntu@sw1:~$ sudo ovs-ofctl -O OpenFlow13 dump-flows sw1
```

Adjunte las salidas. (a) Compare la regla de fwd en sw1 del paso 1 con la del

inciso 2: ¿qué campos nuevos aparecen en el match? ¿Qué ventaja y qué costo tiene incluir las direcciones IP? (b) Complete la tabla con la línea Total del script y el RTT promedio (avg) del resumen de cada ping. (c) ¿Por qué ya no aparecen reglas de fwd en sw1? Explique por qué este modo no permite reenviar a line-rate.

| Modo de fwd | PACKET_IN | PACKET_OUT | FLOW_MOD ADD | RTT promedio (ms) |
|---|---|---|---|---|
| Normal (inciso 2) | 2 | 2 | 6 | 1.326 ms |
| packetOutOnly (paso 3) | 10 | 10 | 0 | 3.802 ms |

**Pista:** en sw1, compare el n_packets de la regla priority=5,ip actions=CONTROLLER del paso 1 con el del paso 4: cada paquete que usa esa regla genera un PACKET_IN en ese switch. Con packetOutOnly, cada solicitud y cada respuesta del ping la usan en todos los switches del camino.

```
# Responder aquí.
```

```
root@controller: # python3 ~/TEL354_LAB5_2026_2/tools/of_resumen.py ~/lab5_pkto.pcap
  t (s)  switch mensaje     detalle
  0.000  sw2    PACKET_IN   h1 -> h3 echo request seq=1 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
  0.006  sw2    PACKET_OUT  h1 -> h3 echo request seq=1 | acción: output:1
  0.009  sw1    PACKET_IN   h1 -> h3 echo request seq=1 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
  0.010  sw1    PACKET_OUT  h1 -> h3 echo request seq=1 | acción: output:3
  0.012  sw3    PACKET_IN   h1 -> h3 echo request seq=1 | reason=ACTION (regla con salida CONTROLLER) | in_port=3
  0.012  sw3    PACKET_OUT  h1 -> h3 echo request seq=1 | acción: output:2
  0.015  sw3    PACKET_IN   h3 -> h1 echo reply seq=1 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
  0.017  sw3    PACKET_OUT  h3 -> h1 echo reply seq=1 | acción: output:3
  0.019  sw1    PACKET_IN   h3 -> h1 echo reply seq=1 | reason=ACTION (regla con salida CONTROLLER) | in_port=3
  0.021  sw1    PACKET_OUT  h3 -> h1 echo reply seq=1 | acción: output:2
  0.023  sw2    PACKET_IN   h3 -> h1 echo reply seq=1 | reason=ACTION (regla con salida CONTROLLER) | in_port=1
  0.024  sw2    PACKET_OUT  h3 -> h1 echo reply seq=1 | acción: output:2
  1.000  sw2    PACKET_IN   h1 -> h3 echo request seq=2 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
  1.002  sw2    PACKET_OUT  h1 -> h3 echo request seq=2 | acción: output:1
  1.004  sw1    PACKET_IN   h1 -> h3 echo request seq=2 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
  1.006  sw1    PACKET_OUT  h1 -> h3 echo request seq=2 | acción: output:3
  1.008  sw3    PACKET_IN   h1 -> h3 echo request seq=2 | reason=ACTION (regla con salida CONTROLLER) | in_port=3
  1.010  sw3    PACKET_OUT  h1 -> h3 echo request seq=2 | acción: output:2
  1.012  sw3    PACKET_IN   h3 -> h1 echo reply seq=2 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
  1.014  sw3    PACKET_OUT  h3 -> h1 echo reply seq=2 | acción: output:3
  1.016  sw1    PACKET_IN   h3 -> h1 echo reply seq=2 | reason=ACTION (regla con salida CONTROLLER) | in_port=3
  1.017  sw1    PACKET_OUT  h3 -> h1 echo reply seq=2 | acción: output:2
  1.018  sw2    PACKET_IN   h3 -> h1 echo reply seq=2 | reason=ACTION (regla con salida CONTROLLER) | in_port=1
  1.019  sw2    PACKET_OUT  h3 -> h1 echo reply seq=2 | acción: output:2
  2.001  sw2    PACKET_IN   h1 -> h3 echo request seq=3 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
  2.004  sw2    PACKET_OUT  h1 -> h3 echo request seq=3 | acción: output:1
  2.005  sw1    PACKET_IN   h1 -> h3 echo request seq=3 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
  2.006  sw1    PACKET_OUT  h1 -> h3 echo request seq=3 | acción: output:3
  2.007  sw3    PACKET_IN   h1 -> h3 echo request seq=3 | reason=ACTION (regla con salida CONTROLLER) | in_port=3
  2.008  sw3    PACKET_OUT  h1 -> h3 echo request seq=3 | acción: output:2
  2.009  sw3    PACKET_IN   h3 -> h1 echo reply seq=3 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
  2.011  sw3    PACKET_OUT  h3 -> h1 echo reply seq=3 | acción: output:3
  2.012  sw1    PACKET_IN   h3 -> h1 echo reply seq=3 | reason=ACTION (regla con salida CONTROLLER) | in_port=3
  2.013  sw1    PACKET_OUT  h3 -> h1 echo reply seq=3 | acción: output:2
```

```
 2.014  sw2    PACKET_IN   h3 -> h1 echo reply seq=3 | reason=ACTION (regla con salida CONTROLLER) | in_port=1
 2.015  sw2    PACKET_OUT  h3 -> h1 echo reply seq=3 | acción: output:2
 3.003  sw2    PACKET_IN   h1 -> h3 echo request seq=4 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
 3.006  sw2    PACKET_OUT  h1 -> h3 echo request seq=4 | acción: output:1
 3.007  sw1    PACKET_IN   h1 -> h3 echo request seq=4 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
 3.009  sw1    PACKET_OUT  h1 -> h3 echo request seq=4 | acción: output:3
 3.011  sw3    PACKET_IN   h1 -> h3 echo request seq=4 | reason=ACTION (regla con salida CONTROLLER) | in_port=3
 3.012  sw3    PACKET_OUT  h1 -> h3 echo request seq=4 | acción: output:2
 3.013  sw3    PACKET_IN   h3 -> h1 echo reply seq=4 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
 3.015  sw3    PACKET_OUT  h3 -> h1 echo reply seq=4 | acción: output:3
 3.016  sw1    PACKET_IN   h3 -> h1 echo reply seq=4 | reason=ACTION (regla con salida CONTROLLER) | in_port=3
 3.018  sw1    PACKET_OUT  h3 -> h1 echo reply seq=4 | acción: output:2
 3.019  sw2    PACKET_IN   h3 -> h1 echo reply seq=4 | reason=ACTION (regla con salida CONTROLLER) | in_port=1
 3.021  sw2    PACKET_OUT  h3 -> h1 echo reply seq=4 | acción: output:2
 4.004  sw2    PACKET_IN   h1 -> h3 echo request seq=5 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
 4.006  sw2    PACKET_OUT  h1 -> h3 echo request seq=5 | acción: output:1
 4.008  sw1    PACKET_IN   h1 -> h3 echo request seq=5 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
 4.009  sw1    PACKET_OUT  h1 -> h3 echo request seq=5 | acción: output:3
 4.010  sw3    PACKET_IN   h1 -> h3 echo request seq=5 | reason=ACTION (regla con salida CONTROLLER) | in_port=3
 4.011  sw3    PACKET_OUT  h1 -> h3 echo request seq=5 | acción: output:2
 4.012  sw3    PACKET_IN   h3 -> h1 echo reply seq=5 | reason=ACTION (regla con salida CONTROLLER) | in_port=2
 4.014  sw3    PACKET_OUT  h3 -> h1 echo reply seq=5 | acción: output:3
 4.015  sw1    PACKET_IN   h3 -> h1 echo reply seq=5 | reason=ACTION (regla con salida CONTROLLER) | in_port=3
 4.016  sw1    PACKET_OUT  h3 -> h1 echo reply seq=5 | acción: output:2
 4.017  sw2    PACKET_IN   h3 -> h1 echo reply seq=5 | reason=ACTION (regla con salida CONTROLLER) | in_port=1
 4.018  sw2    PACKET_OUT  h3 -> h1 echo reply seq=5 | acción: output:2

Total (mensajes del ping y FLOW_MOD): PACKET_IN=30, PACKET_OUT=30, FLOW_MOD ADD=0, FLOW_MOD DELETE=0
root@controller:~#
```

a. Comparación de la regla de fwd en sw1 (paso 1 vs inciso 2):
   - Aparecen nuevos campos de capa 3 como ip, nw_src=10.0.0.1, nw_dst=10.0.0.3. Además, se permite un control de tráfico granular basado en direcciones IP, aislando flujos entre diferentes servicios IP dentro de un mismo host. Asimismo, aumenta exponencialmente la cantidad de entradas requeridas en la tabla TCAM del switch, lo que puede agotar rápidamente la memoria de hardware si existen múltiples flujos activos.

b. Respondido

c. ¿Por qué ya no aparecen reglas de fwd en sw1 y por qué no permite reenviar a line-rate?
   - No aparecen reglas específicas porque packetOutOnly deshabilita la emisión de mensajes FLOW_MOD ADD. Cada uno de los 5 Echo Requests y 5 Echo Replies coincide únicamente con la regla base priority=5, ip actions=CONTROLLER, enviando todos los paquetes al controlador. Asimismo, no permite reenviar a line-rate porque ningún paquete se conmuta en el hardware del switch, ya que el total del tráfico debe atravesar el plano de control elevando así la latencia promedio por paquete.

## ACTIVIDAD 3: CONTROL DE ACCESO REACTIVO CON INTENTS (RBAC) (1.0 pto)

En esta actividad la decisión de cada PACKET_IN la toma una aplicación externa en Python (rbac_app.py), según una política por roles (policy.yaml). La app de ONOS packet-bridge es el puente: recibe los PACKET_IN de IPv4 y se los envía por HTTP a rbac_app.py.

**Archivos:** están en la carpeta rbac/ del repositorio del laboratorio (https://github.com/NohaFeynman/TEL354_LAB5_2026_2/tree/main/rbac), que ya descargó en el controller (~/TEL354_LAB5_2026_2/rbac, actualizado con git pull en la Actividad 0):

| Archivo | Función |
|---|---|
| packet-bridge-1.0.0.oar | App de ONOS (Java): solicita los paquetes IPv4 y reenvía cada PACKET_IN por HTTP a rbac_app.py. |
| rbac_app.py | Aplicación en Python: decide con la política y crea un HostToHostIntent o una regla DROP por la API REST de ONOS. Su código completo está en el Anexo A. |
| policy.yaml | Política: rol de cada host y comunicaciones permitidas entre roles. Su contenido se muestra a continuación. |

**Contenido de policy.yaml:**

```
# Política de control de acceso por roles (RBAC).
# hosts: rol de cada host según su IP en la red de datos.
# rules: qué rol de origen puede comunicarse con qué rol de destino.
#        Lo que no aparece en rules se deniega (regla DROP temporal).
hosts:
  10.0.0.1: {name: h1, role: administrador}
  10.0.0.2: {name: h2, role: alumno}
  10.0.0.3: {name: h3, role: servidor}
  10.0.0.4: {name: h4, role: invitado}

rules:
  - {src: administrador, dst: servidor, allow: true}
  - {src: alumno,        dst: servidor, allow: true}
  - {src: invitado,      dst: servidor, allow: false}
```

1. **(0.25 ptos)** Instalar packet-bridge e iniciar la aplicación RBAC. Ejecute los pasos en este orden:

**Paso 1 (CLI de ONOS):** desactive fwd, para que no compita con packet-bridge por los mismos paquetes:

```
onos@root > app deactivate org.onosproject.fwd
```

**Paso 2 (controller, terminal 1):** revise la política e instale packet-bridge por la API REST:

```
ubuntu@controller:~$ cd ~/TEL354_LAB5_2026_2/rbac && cat policy.yaml
ubuntu@controller:~$ curl -s -u onos:rocks -X POST -o /dev/null \
  -w "HTTP %{http_code}\n" -H 'Content-Type: application/octet-stream' \
  --data-binary @packet-bridge-1.0.0.oar \
  'http://localhost:8181/onos/v1/applications?activate=true'
```

**Paso 3 (CLI de ONOS y sw1):** verifique que la app esté activa y revise las reglas de sw1:

```
onos@root > apps -s | grep -i bridge
onos@root > packet-processors
ubuntu@sw1:~$ sudo ovs-ofctl -O OpenFlow13 dump-flows sw1
```

**Paso 4 (controller, terminal 3 nueva):** inicie la aplicación RBAC y déjela abierta durante toda la actividad. Allí verá cada PACKET_IN y la decisión tomada:

```
ubuntu@controller:~$ python3 ~/TEL354_LAB5_2026_2/rbac/rbac_app.py
```

**Verificación:** el POST debe responder HTTP 200, apps -s debe listar edu.pucp.sdn.packetbridge y la app debe imprimir los cuatro hosts con su rol.

Adjunte las salidas. ¿Qué regla nueva aparece en sw1 y para qué sirve? ¿Qué rol tiene cada host y qué comunicaciones permite la política?

**Pista:** compare las reglas de sw1 con las del inciso 1 de la Actividad 2 (las 3 reglas base de ONOS). La regla nueva es la que hace match con ip y tiene como acción CONTROLLER: es la que solicitó packet-bridge al PacketService.

```
# Responder aquí.
Salidas:
```

```
root@controller:~# cd ~/TEL354_LAB5_2026_2/rbac && cat policy.yaml
# Política de control de acceso por roles (RBAC).
# hosts: rol de cada host según su IP en la red de datos.
# rules: qué rol de origen puede comunicarse con qué rol de destino.
#        Lo que no aparece en rules se deniega (regla DROP temporal).
hosts:
  10.0.0.1: {name: h1, role: administrador}
  10.0.0.2: {name: h2, role: alumno}
  10.0.0.3: {name: h3, role: servidor}
  10.0.0.4: {name: h4, role: invitado}

rules:
  - {src: administrador, dst: servidor, allow: true}
  - {src: alumno,        dst: servidor, allow: true}
  - {src: invitado,      dst: servidor, allow: false}
root@controller:~/TEL354_LAB5_2026_2/rbac# curl -s -u onos:rocks -X POST -o /dev/null \
  -w "HTTP %{http_code}\n" -H 'Content-Type: application/octet-stream' \
  --data-binary @packet-bridge-1.0.0.oar \
  'http://localhost:8181/onos/v1/applications?activate=true'
HTTP 200
root@controller:~/TEL354_LAB5_2026_2/rbac# python3 ~/TEL354_LAB5_2026_2/rbac/rbac_app.py
01:48:08  RBAC escuchando en http://127.0.0.1:5000/packet-in
01:48:08  Política: /root/TEL354_LAB5_2026_2/rbac/policy.yaml
01:48:08              h1  10.0.0.1   rol=administrador
01:48:08              h2  10.0.0.2   rol=alumno
01:48:08              h3  10.0.0.3   rol=servidor
01:48:08              h4  10.0.0.4   rol=invitado
```

```
onos@root > app deactivate org.onosproject.fwd                                  01:44:32
Deactivated org.onosproject.fwd
onos@root > apps -s | grep -i bridge                                            01:44:36
* 176 edu.pucp.sdn.packetbridge          1.0.0    Campus RBAC Packet Bridge
onos@root > packet-processors                                                   01:46:47
priority=advisor(0), class=org.onosproject.provider.lldp.impl.LldpLinkProvider$InternalPacketProcessor, packets=219704, avgNanos=213502
priority=advisor(1), class=org.onosproject.provider.host.impl.HostLocationProvider$InternalHostProvider, packets=222802, avgNanos=26488
priority=advisor(4), class=edu.pucp.sdn.packetbridge.PacketBridgeProcessor, packets=48, avgNanos=6316
priority=director(1), class=org.onosproject.net.neighbour.impl.NeighbourResolutionManager$InternalPacketProcessor, packets=222637, avgNanos=1657
onos@root >                                                                     01:47:07
```

```
root@sw1:~# ovs-ofctl -O OpenFlow13 dump-flows sw1
 cookie=0x1000054b0daab, duration=172676.114s, table=0, n_packets=111402, n_bytes=15150672, send_flow_rem priority=40000,dl_type=0x8942 actions=CONTROLLER:655
35,clear_actions
 cookie=0x1000095ba66d0, duration=172676.108s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=40000,dl_type=0x88cc actions=CONTROLLER:65535,clear_act
ions
 cookie=0x10000607dcbc0, duration=172676.078s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=40000,arp actions=CONTROLLER:65535,clear_actions
 cookie=0x100003b5d6ece, duration=66.105s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=5,ip actions=CONTROLLER:65535,clear_actions
root@sw1:~#
```

- Vemos que aparece la regla priority=5, ip actions=CONTROLLER:65535, la cual es solicitada por packet-bridge al PacketService de ONOS para que cualquier paquete IPv4 que ingrese sea enviado al controlador y redirigido por HTTP hacia la aplicación externa rbac_app.py.
- Roles de cada host + permisos de la política:

h1 (10.0.0.1): administrador → Tiene acceso permitido al servidor.
h2 (10.0.0.2): alumno → Tiene acceso permitido al servidor.
h3 (10.0.0.3): servidor.
h4 (10.0.0.4): invitado → Denegado hacia el servidor.

2. **(0.5 ptos)** Tráfico permitido y denegado. Ejecute los pasos en este orden y observe la terminal 3 después de cada ping:

**Paso 1 (h1, administrador):** envíe 5 pings al servidor (h3):

```
ubuntu@h1:~$ ping -c 5 10.0.0.3
```

**Paso 2 (h4, invitado):** envíe 5 pings al servidor:

```
ubuntu@h4:~$ ping -c 5 10.0.0.3
```

**Paso 3 (CLI de ONOS y switches):** revise los intents y las reglas de los tres switches:

```
onos@root > intents
ubuntu@swX:~$ sudo ovs-ofctl -O OpenFlow13 dump-flows $(hostname)
```

**Paso 4 (controller, terminal 1):** consulte a ONOS la ruta entre h1 y h3 con el recurso /paths (equivalente a /wm/topology/route de Floodlight):

```
ubuntu@controller:~$ curl -s -u onos:rocks http://localhost:8181/onos/v1/hosts |
python3 -m json.tool
ubuntu@controller:~$ curl -s -u onos:rocks
"http://localhost:8181/onos/v1/paths/MAC_H1%2FNone/MAC_H3%2FNone" | python3 -m
json.tool
```

**Nota:** el id de un host tiene la forma MAC/None; en la URL la barra se escribe %2F. La respuesta lista los enlaces (src → dst) del camino más corto. Reemplazar MAC_H1 y MAC_H3 por los registros cuyo campo ipAddresses contenga 10.0.0.1 y 10.0.0.3, y copiar sus respectivos campos id (de la consulta /onos/v1/hosts).

**Paso 5 (controller, terminal 1):** revise en el código qué hace la app con cada decisión:

```
ubuntu@controller:~$ grep -n -E "def |Intent|priority|TIMEOUT" \
  ~/TEL354_LAB5_2026_2/rbac/rbac_app.py
```

```
ubuntu@controller:~$ python3 ~/TEL354_LAB5_2026_2/rbac/rbac_app.py
22:05:06  RBAC escuchando en http://127.0.0.1:5000/packet-in  (política: /home/ubuntu/TEL354_LAB5_2026_2/rbac/policy.yaml)
22:05:06             h1  10.0.0.1   rol=administrador
22:05:06             h2  10.0.0.2   rol=alumno
22:05:06             h3  10.0.0.3   rol=servidor
22:05:06             h4  10.0.0.4   rol=invitado
22:06:59  [PACKET_IN] of:00001a749039894a puerto 2  h1 (10.0.0.1) -> h3 (10.0.0.3)  proto=1
22:07:00  [ALLOW]     administrador -> servidor: HostToHostIntent key=0
22:07:39  [PACKET_IN] of:0000ca272d265744 puerto 1  h4 (10.0.0.4) -> h3 (10.0.0.3)  proto=1
22:07:39  [DENY]      invitado -> servidor: regla DROP en of:0000ca272d265744 por 30 s
```

*Figura 3. Decisiones de la aplicación RBAC.*

Adjunte los dos pings, la terminal 3, la salida de intents y las reglas de cada switch. Responda: (a) Cada línea [PACKET_IN] de la terminal 3 es un PACKET_IN de IPv4 que ONOS recibió de cualquier switch; el valor of:... indica

cuál (compárelo con la salida de devices). ¿Cuántas líneas [PACKET_IN] generó cada ping y de qué switch vinieron? ¿Por qué el ping de h1 perdió solo el primer paquete y ningún otro switch del camino envió PACKET_IN? (b) ¿Qué creó la app para h1 → h3? ¿En qué switches aparecen sus reglas? ¿Coinciden con el camino que devolvió /paths? ¿Por qué los paquetes siguientes ya no llegan al controlador? (c) Para h4 → h3, ¿qué regla se instaló, en qué switch, con qué prioridad y por cuánto tiempo? ¿Por qué el ping de h4 generó un solo PACKET_IN aunque se perdieron los 5 paquetes?

**Pista:** (a) packet-bridge descarta el paquete original y no envía un PACKET_OUT; compárelo con fwd (Actividad 2), que sí lo hace. Puede comprobar el número de PACKET_IN con el n_packets de la regla priority=5,ip de cada switch: solo aumenta en el switch de acceso. (b) Las reglas del intent son las que hacen match con las MAC de h1 y h3; búsquelas en los tres switches. (c) La regla DROP es la de prioridad 40001 en el switch donde está conectado h4 (revise el device_id del [PACKET_IN] en la terminal 3 y la salida de devices); una regla sin acciones descarta el paquete en el switch.

```
# Responder aquí.
```

```
root@h1:~# ping -c 5 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=8.91 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=2.05 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=1.84 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=1.81 ms

--- 10.0.0.3 ping statistics ---
5 packets transmitted, 4 received, 20% packet loss, time 4060ms
rtt min/avg/max/mdev = 1.812/3.650/8.908/3.036 ms
root@h1:~#
```

```
root@h4:~# ping -c 5 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.

--- 10.0.0.3 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4132ms
```

```
onos@root > intents
Id: 0x0
State: INSTALLED
Key: 0x0
Intent type: HostToHostIntent
Application Id: edu.pucp.sdn.packetbridge
Leader Id: 172.17.0.1
Resources: [FA:16:3E:88:05:79/None, FA:16:3E:05:4C:50/None]
Treatment: [NOACTION]
Constraints: [LinkTypeConstraint{inclusive=false, types=[OPTICAL]}]
Source host: FA:16:3E:88:05:79/None
Destination host: FA:16:3E:05:4C:50/None
```

```
root@sw1:~# ovs-ofctl -O OpenFlow13 dump-flows sw1
 cookie=0x1000054b0daab, duration=173984.696s, table=0, n_packets=112248, n_bytes=15265728, send_flow_rem priority=40000,dl_type=0x8942 actions=CONTROLLER:655
35,clear_actions
 cookie=0x1000095ba66d0, duration=173984.690s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=40000,dl_type=0x88cc actions=CONTROLLER:65535,clear_act
ions
 cookie=0x10000607dcbc0, duration=173984.660s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=40000,arp actions=CONTROLLER:65535,clear_actions
 cookie=0x100003b5d6ece, duration=1374.687s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=5,ip actions=CONTROLLER:65535,clear_actions
 cookie=0x940000bf30d819, duration=294.435s, table=0, n_packets=4, n_bytes=392, send_flow_rem priority=100,in_port=ens5,dl_src=fa:16:3e:88:05:79,dl_dst=fa:16:
3e:05:4c:50 actions=output:ens6
 cookie=0x940000a95a3588, duration=294.435s, table=0, n_packets=4, n_bytes=392, send_flow_rem priority=100,in_port=ens6,dl_src=fa:16:3e:05:4c:50,dl_dst=fa:16:
3e:88:05:79 actions=output:ens5
root@sw1:~#
```

```
root@controller:~# curl -s -u onos:rocks http://localhost:8181/onos/v1/hosts | python3 -m json.tool
{
    "hosts": [
        {
            "id": "FA:16:3E:41:9B:6D/None",
            "mac": "FA:16:3E:41:9B:6D",
            "vlan": "None",
            "innerVlan": "None",
            "outerTpid": "0x0000",
            "configured": false,
            "suspended": false,
            "ipAddresses": [
                "10.0.0.4"
            ],
            "locations": [
                {
                    "elementId": "of:0000ca272d265744",
                    "port": "1"
                }
            ]
        },
        {
            "id": "FA:16:3E:88:05:79/None",
            "mac": "FA:16:3E:88:05:79",
            "vlan": "None",
            "innerVlan": "None",
            "outerTpid": "0x0000",
            "configured": false,
            "suspended": false,
            "ipAddresses": [
                "10.0.0.1"
            ],
            "locations": [
                {
                    "elementId": "of:00001a749039894a",
                    "port": "2"
```

```
            "locations": [
                {
                    "elementId": "of:00001a749039894a",
                    "port": "2"
                }
            ]
        },
        {
            "id": "FA:16:3E:05:4C:50/None",
            "mac": "FA:16:3E:05:4C:50",
            "vlan": "None",
            "innerVlan": "None",
            "outerTpid": "0x0000",
            "configured": false,
            "suspended": false,
            "ipAddresses": [
                "10.0.0.3"
            ],
            "locations": [
                {
                    "elementId": "of:0000ca272d265744",
                    "port": "2"
                }
            ]
        }
    ]
}
root@controller:~# curl -s -u onos:rocks "http://localhost:8181/onos/v1/paths/MAC_H1%2FNone/MAC_H3%2FNone" | python3 -m json.tool
{
    "paths": []
}
```

```
root@controller:~# grep -n -E "def |Intent|priority|TIMEOUT" \
  ~/TEL354_LAB5_2026_2/rbac/rbac_app.py
8:    - permitido: crea un HostToHostIntent entre ambos hosts;
29:DROP_TIMEOUT = 30                    # segundos de vida de una regla DROP
37:def log(msg):
41:def onos(method, path, body=None):
52:def decide(src_ip, dst_ip):
64:def name(ip):
68:def allow(p, why):
69:    """Permitido: HostToHostIntent entre las MAC del PACKET_IN."""
71:        'type': 'HostToHostIntent', 'appId': APP_ID,
76:    log(f'[ALLOW]     {why}: HostToHostIntent key={key}')
79:def deny(p, why):
83:        'priority': 40001, 'timeout': DROP_TIMEOUT,
90:    log(f'[DENY]      {why}: regla DROP en {dev} por {DROP_TIMEOUT} s')
92:    threading.Timer(DROP_TIMEOUT, decided.pop, (pair, None)).start()
95:def handle(p):
116:    def do_POST(self):
127:    def log_message(self, *args):
root@controller:~#
```

a. Análisis de los PACKET_IN:

- Ping h1 -> h3: Generó 1 solo PACKET_IN originado en el switch de ingreso de h1 (sw2, of:...).
- Ping h4 -> h3: Generó 1 solo PACKET_IN originado en el switch de ingreso de h4 (sw3, of:...).
- Pérdida del primer paquete en h1: packet-bridge descarta el paquete de datos original tras enviar la notificación HTTP a la app Python. El HostToHostIntent instala las reglas en el camino en respuesta a ese primer paquete, por lo que a partir del segundo paquete la comunicación fluye directamente a velocidad de línea.

b. Acción para h1 -> h3:

- La app creó un HostToHostIntent bidireccional entre FA:16:3E:88:05:79/None y FA:16:3E:05:4C:50/None.
- Vemos que aparecen reglas compiladas en los 3 switches de la ruta (sw2, sw1, sw3), coincidiendo perfectamente con la topología que devuelve /paths.
- Los paquetes subsiguientes no llegan al controlador porque hacen match directo con las reglas de alta prioridad (priority=100) instaladas en el datapath.

c. Acción para h4 -> h3:

- Se instaló una regla de DROP (treatment: {instructions: []}) únicamente en el switch de ingreso de h4 (sw3), con prioridad 40001 y un temporizador de 30 segundos.
- Generó un solo PACKET_IN porque la regla DROP de prioridad 40001 intercepta en el switch los 4 paquetes siguientes antes de que alcancen la regla de priority=5, descartándolos directamente en el hardware sin molestar al controlador.

3. **(0.25 ptos)** Cambio de política. Ejecute los pasos en este orden:

**Paso 1 (terminal 3):** detenga la app con Ctrl+C, permita el acceso del invitado y

vuelva a iniciarla (la app lee policy.yaml al arrancar):

```
ubuntu@controller:~$ sed -i 's/allow: false/allow: true/' \
  ~/TEL354_LAB5_2026_2/rbac/policy.yaml
ubuntu@controller:~$ grep invitado ~/TEL354_LAB5_2026_2/rbac/policy.yaml
ubuntu@controller:~$ python3 ~/TEL354_LAB5_2026_2/rbac/rbac_app.py
```

**Paso 2 (h4):** espere a que expire la regla DROP (30 s sin tráfico desde el paso 2 del inciso anterior, más unos segundos hasta que ONOS la borra) y repita el ping:

```
ubuntu@h4:~$ sleep 40; ping -c 5 10.0.0.3
onos@root > intents
```

**Verificación:** la terminal 3 debe mostrar un [PACKET_IN] de h4 seguido de [ALLOW], y intents debe listar un segundo intent.

Adjunte las salidas. ¿Qué cambió en la red sin configurar ningún switch? Compare esta app con fwd: ¿quién toma la decisión, qué reglas resultan y qué pasa con los paquetes siguientes al primero?

**Pista:** el ping se repite después de esperar porque, mientras la regla DROP exista, el switch descarta los paquetes de h4 sin generar PACKET_IN y la app nunca se entera del cambio de política.

```
# Responder aquí.
```

```
root@h4:~# sleep 40; ping -c 5 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=6.33 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=1.06 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=0.959 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=1.02 ms

--- 10.0.0.3 ping statistics ---
5 packets transmitted, 4 received, 20% packet loss, time 4009ms
rtt min/avg/max/mdev = 0.959/2.342/6.334/2.304 ms
root@h4:~#
```

```
onos@root > intents
Id: 0x5
State: INSTALLED
Key: 0x5
Intent type: HostToHostIntent
Application Id: edu.pucp.sdn.packetbridge
Leader Id: 172.17.0.1
Resources: [FA:16:3E:41:9B:6D/None, FA:16:3E:05:4C:50/None]
Treatment: [NOACTION]
Constraints: [LinkTypeConstraint{inclusive=false, types=[OPTICAL]}]
Source host: FA:16:3E:41:9B:6D/None
Destination host: FA:16:3E:05:4C:50/None

Id: 0x0
State: INSTALLED
Key: 0x0
Intent type: HostToHostIntent
Application Id: edu.pucp.sdn.packetbridge
Leader Id: 172.17.0.1
Resources: [FA:16:3E:88:05:79/None, FA:16:3E:05:4C:50/None]
Treatment: [NOACTION]
Constraints: [LinkTypeConstraint{inclusive=false, types=[OPTICAL]}]
Source host: FA:16:3E:88:05:79/None
Destination host: FA:16:3E:05:4C:50/None
```

```
root@controller:~/TEL354_LAB5_2026_2/rbac# python3 ~/TEL354_LAB5_2026_2/rbac/rbac_app.py
02:28:04  RBAC escuchando en http://127.0.0.1:5000/packet-in
02:28:04  Política: /root/TEL354_LAB5_2026_2/rbac/policy.yaml
02:28:04             h1  10.0.0.1   rol=administrador
02:28:04             h2  10.0.0.2   rol=alumno
02:28:04             h3  10.0.0.3   rol=servidor
02:28:04             h4  10.0.0.4   rol=invitado
02:30:57  [PACKET_IN] of:0000ca272d265744 puerto 1  h4 (10.0.0.4) -> h3 (10.0.0.3)  proto=1
02:30:57  [ALLOW]     invitado -> servidor: HostToHostIntent key=5
```

a. Vemos que cambió dinámicamente el comportamiento de acceso de la red … al expirar la regla DROP previa en sw3, el siguiente paquete de h4 generó un nuevo PACKET_IN. Para este caso, la app evaluó el archivo policy.yaml actualizado y emitió un HostToHostIntent a ONOS, permitiendo la comunicación sin haber tocado directamente la CLI de los switches.

b. Comparación entre esta app y fwd:

Respecto a la toma de decisión, en forwarding, la decisión la toma internamente ONOS conectando L2/L3 según topología. En RBAC, la decisión la toma un componente externo mediante políticas de control de acceso orientadas a roles.

En relación a las reglas resultantes, fwd instala flujos L2/L3 de bajo nivel con idle_timeout corto (10s) en cada switch. Asimismo, RBAC abstrae la conectividad mediante Intents de alto nivel cuando se aprueba el acceso, o instala reglas de descarte temporal de alta prioridad en la frontera cuando se deniega.

En relación a los paquetes subsiguientes, en ambos casos permitidos, los paquetes siguientes se conmutan en el datapath a line-rate. Notamos que en el caso denegado de RBAC, la regla DROP detiene el tráfico en el primer switch evitando la saturación del plano de control.

## ACTIVIDAD 4: COMPARACIÓN Y CONCLUSIONES (1.0 pto)

1. **(0.5 ptos)** Complete la tabla con sus evidencias. Use una línea por celda; si un valor no lo midió, escriba el valor esperado y marque "(esperado)".

**Dónde encontrar cada dato:** proactivo, Actividad 4 del informe previo (intent creado por REST); fwd, incisos 2 y 3 de la Actividad 2; fwd con packetOutOnly, inciso 4 de la Actividad 2; RBAC, inciso 2 de la Actividad 3.

| Criterio | Proactivo (intent por REST) | Reactivo fwd | Reactivo fwd packetOutOnly | Reactivo RBAC (intents) |
|---|---|---|---|---|
| Quién calcula la ruta y en qué momento | ONOS (IntentService) de forma anticipada al recibir la solicitud REST del administrador. | ONOS (fwd) en tiempo real al recibir el primer PACKET_IN del flujo. | ONOS (fwd) paquete por paquete al recibir cada PACKET_IN. | ONOS (IntentService) de forma reactiva tras la aprobación de la app Python ante el primer PACKET_IN. |
| Quién decide permitir o denegar | El operador/administrador al invocar la API REST para crear el Intent. | La aplicación fwd de ONOS (permite todo el tráfico IP por defecto). | La aplicación fwd de ONOS (reenvía todo tráfico mediante PACKET_OUT). | La aplicación externa Python (rbac_app.py) consultando policy.yaml. |
| PACKET_IN de IPv4 en un ping de 5 paquetes | 0 paquetes (reglas preinstaladas en el datapath) (esperado). | 2 paquetes (1 para Request en origen, 1 para Reply en destino). | 10 paquetes (5 Request + 5 Reply, 1 por paquete enviado al controlador). | 1 paquete (solo el primer Request en el switch de ingreso; el Intent es bidireccional). |
| RTT del primer paquete / de los siguientes | ~0.80 ms / ~0.80 ms (esperado) | 3.24 ms / ~0.85 ms | 3.85 ms / ~3.80 ms | Perdió el 1er paquete (descarte por packet-bridge) / ~0.85 ms |
| Switches con reglas, prioridad y timeout | Todos los switches de la ruta, prioridad por defecto, sin timeout (permanente). | Todos los switches de la ruta, priority=10, idle_timeout=10s (o 60s). | Ningún switch tiene reglas específicas (solo regla base priority=5). | Si es permitido: todos los del camino (Intent). Si es denegado: solo switch de ingreso, priority=40001, timeout=30s. |
| ¿Los paquetes siguientes se reenvían a line-rate? | Sí, conmutación en el datapath del switch. | Sí, mientras la regla no expire por idle_timeout. | No, el 100% del tráfico atraviesa el plano de control. | Sí, conmutados a line-rate por el Intent (o descartados a line-rate por la regla DROP). |

**Ejemplo:** para "Quién decide permitir o denegar" en la columna proactivo, una respuesta válida es "el operador, al crear o no el intent".

2. **(0.5 ptos)** Conclusiones. ¿Qué enfoque usaría en el proyecto del curso y por qué? Considere la latencia del primer paquete, la carga del controlador con muchos flujos nuevos por segundo, el reenvío a line-rate y la necesidad de

aplicar una política de acceso. Redacte dos conclusiones técnicas sustentadas en sus evidencias.

```
# Responder aquí.
Conclusiones:
    • El uso de esquemas reactivos híbridos mediante la abstracción de Intents, como en el caso de RBAC, o la instalación reactiva de reglas de flujo ofrece un equilibrio óptimo entre el control granular centralizado y la escalabilidad de la red. Frente al modo packetOutOnly, el cual degrada el rendimiento al enviar el total de los paquetes al controlador (RTT de ~3.8 ms) y satura el canal OpenFlow, la delegación de decisiones al datapath tras el primer PACKET_IN permite la conmutación de los paquetes subsecuentes a velocidad de línea (aprox. line-rate con RTT de ~0.85 ms), reduciendo así la carga en la CPU del controlador ONOS.
    • La instalación de reglas temporales de descarte de alta prioridad directamente en el switch de acceso ante solicitudes no autorizadas (RBAC) es un mecanismo eficaz de seguridad y protección. Al absorber y filtrar el tráfico denegado en la interfaz de hardware del primer switch, se evita que las peticiones maliciosas o los escaneos continuos generen múltiples eventos PACKET_IN, aislando y protegiendo así de esta forma la infraestructura del plano de control frente a ataques de Denegación de Servicio.
```

**Al terminar,** deje el slice en su estado inicial:

```
ubuntu@controller:~$ cd ~/TEL354_LAB5_2026_2/ansible
ubuntu@controller:~$ ansible-playbook slice_reset.yml
```

## REFERENCIAS


- ONOS Project / Open Networking Foundation. Documentación de ONOS y µONOS. https://docs.onosproject.org
- ONOS classic wiki (copia comunitaria de wiki.onosproject.org): Appendix B: REST API. https://andrea-campanella.github.io/onos-classic-wiki/onos/guides/developer-guide/appendix-b-rest-api/
- ONOS Project. Código fuente de ONOS (incluye los recursos de la API REST). https://github.com/opennetworkinglab/onos
- Open Networking Foundation, “OpenFlow Switch Specification”, versión 1.3.5, 2015. https://opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.3.5.pdf
- Ansible Documentation. Getting started. https://docs.ansible.com/ansible/latest/

## Criterios de evaluación del laboratorio

| N° | Actividad | Puntaje |
|---|---|---|
| 0 | Preparación | 0 ptos |
| 1 | ONOS contenerizado y bootstrap de la red | 1.5 ptos |
| 2 | Enfoque reactivo con la aplicación fwd | 1.5 ptos |
| 3 | Control de acceso reactivo con intents (RBAC) | 1.0 pto |
| 4 | Comparación y conclusiones | 1.0 pto |
| | **Total** | **5 ptos** |

## ANEXO A: CÓDIGO DE rbac_app.py

Archivo ~/TEL354_LAB5_2026_2/rbac/rbac_app.py (también en la carpeta rbac/ del repositorio). Las funciones decide(), allow() y deny() contienen la lógica que analizará en la Actividad 3.

```
#!/usr/bin/env python3
"""Control de acceso por roles (RBAC) reactivo sobre ONOS.

1. Un switch envía un PACKET_IN de IPv4 a ONOS.
2. packet-bridge (app Java dentro de ONOS) lo reenvía por HTTP a
   POST http://127.0.0.1:5000/packet-in y descarta el paquete.
3. Esta app consulta policy.yaml y decide:
   - permitido: crea un HostToHostIntent entre ambos hosts;
   - denegado: instala una regla DROP temporal en el switch de ingreso.

Uso: python3 rbac_app.py   (Ctrl+C para detener)
"""
import base64
import json
import os
import threading
import urllib.error
import urllib.request
from datetime import datetime
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

import yaml

ONOS = os.getenv('ONOS_URL', 'http://127.0.0.1:8181') + '/onos/v1'
AUTH = 'Basic ' + base64.b64encode(b'onos:rocks').decode()
APP_ID = 'edu.pucp.sdn.packetbridge'   # appId registrado por packet-bridge
HERE = os.path.dirname(os.path.abspath(__file__))
POLICY = os.path.join(HERE, 'policy.yaml')
DROP_TIMEOUT = 30                     # segundos de vida de una regla DROP

policy = yaml.safe_load(open(POLICY))
hosts = policy['hosts']               # IP -> {name, role}
lock = threading.Lock()
decided = {}                          # (ip_src, ip_dst) -> decisión


def log(msg):
    print(f'{datetime.now():%H:%M:%S}  {msg}', flush=True)


def onos(method, path, body=None):
    """Llama a la API REST de ONOS; devuelve (código, Location)."""
    data = json.dumps(body).encode() if body else None
    req = urllib.request.Request(
        ONOS + path, method=method, data=data,
        headers={'Authorization': AUTH,
                 'Content-Type': 'application/json'})
```

```
    with urllib.request.urlopen(req, timeout=5) as r:
        return r.status, r.headers.get('Location', '')


def decide(src_ip, dst_ip):
    """Devuelve (permitido, motivo) según los roles de origen y destino."""
    src, dst = hosts.get(src_ip), hosts.get(dst_ip)
    if not src or not dst:
        return False, 'host desconocido'
    roles = f"{src['role']} -> {dst['role']}"
    for rule in policy['rules']:
        if rule['src'] == src['role'] and rule['dst'] == dst['role']:
            return rule['allow'], roles
    return False, roles + ' (sin regla: denegado por defecto)'


def name(ip):
    return hosts.get(ip, {}).get('name', ip)


def allow(p, why):
    """Permitido: HostToHostIntent entre las MAC del PACKET_IN."""
    _, loc = onos('POST', '/intents', {
        'type': 'HostToHostIntent', 'appId': APP_ID,
        'one': f"{p['eth_src']}/None", 'two': f"{p['eth_dst']}/None"})
    with lock:   # el intent es bidireccional: vale también para la vuelta
        decided[(p['dst_ip'], p['src_ip'])] = 'ALLOW'
    key = loc.rsplit('/', 1)[-1]
    log(f'[ALLOW]     {why}: HostToHostIntent key={key}')


def deny(p, why):
    """Denegado: regla DROP temporal en el switch de ingreso."""
    dev = p['device_id']
    onos('POST', f'/flows/{dev}?appId={APP_ID}', {
        'priority': 40001, 'timeout': DROP_TIMEOUT,
        'isPermanent': False, 'deviceId': dev,
        'selector': {'criteria': [
            {'type': 'ETH_TYPE', 'ethType': '0x0800'},
            {'type': 'IPV4_SRC', 'ip': p['src_ip'] + '/32'},
            {'type': 'IPV4_DST', 'ip': p['dst_ip'] + '/32'}]},
        'treatment': {'instructions': []}})   # sin acciones = descartar
    log(f'[DENY]      {why}: regla DROP en {dev} por {DROP_TIMEOUT} s')
    pair = (p['src_ip'], p['dst_ip'])
    threading.Timer(DROP_TIMEOUT, decided.pop, (pair, None)).start()

def handle(p):
    pair = (p['src_ip'], p['dst_ip'])
    log(f"[PACKET_IN] {p['device_id']} puerto {p['in_port']}  "
        f"{name(pair[0])} ({pair[0]}) -> {name(pair[1])} ({pair[1]})  "
        f"proto={p['ip_proto']}")
    with lock:
        if pair in decided:
            log(f'            decisión ya aplicada: {decided[pair]}')
            return
```

```
        ok, why = decide(*pair)
        decided[pair] = 'ALLOW' if ok else 'DENY'
    try:
        allow(p, why) if ok else deny(p, why)
    except urllib.error.HTTPError as e:
        log(f'[ERROR]     ONOS respondió HTTP {e.code}: '
            f'{e.read().decode()[:200]}')
        with lock:
            decided.pop(pair, None)


class Handler(BaseHTTPRequestHandler):
    def do_POST(self):
        if self.path != '/packet-in':
            self.send_response(404)
            self.end_headers()
            return
        size = int(self.headers.get('Content-Length', 0))
        p = json.loads(self.rfile.read(size))
        self.send_response(200)        # responde rápido a packet-bridge
        self.end_headers()
        threading.Thread(target=handle, args=(p,), daemon=True).start()

    def log_message(self, *args):
        pass


if __name__ == '__main__':
    log(f'RBAC escuchando en http://127.0.0.1:5000/packet-in')
    log(f'Política: {POLICY}')
    for ip, h in hosts.items():
        log(f"            {h['name']:3} {ip:10} rol={h['role']}")
    ThreadingHTTPServer(('127.0.0.1', 5000), Handler).serve_forever()
```

## ANEXO B: API REST DE ONOS 2.7 (REFERENCIA PARA EL PROYECTO)

Todos los recursos usan la base http://<IP_CONTROLLER>:8181/onos/v1/ y autenticación básica (onos/rocks). La lista completa, con los parámetros y ejemplos de cada recurso, está en la documentación interactiva de su propio controlador (http://localhost:8181/onos/v1/docs/ con el túnel SSH del informe previo). La columna Floodlight indica el recurso equivalente que usó en el laboratorio 4.

| Función | ONOS 2.7 (/onos/v1) | Floodlight (equivalente) |
|---|---|---|
| Switches conectados | GET /devices | GET /wm/core/controller/switches/json |
| Puertos de un switch | GET /devices/{deviceId}/ports | GET /wm/core/switch/{dpid}/port/json |
| Enlaces descubiertos | GET /links | GET /wm/topology/links/json |
| Hosts descubiertos | GET /hosts | GET /wm/device/ |
| Ruta entre dos elementos | GET /paths/{src}/{dst} (switches o hosts) | GET /wm/topology/route/{src-dpid}/{src-port}/{dst-dpid}/{dst-port}/json |
| Instalar una regla (static entry) | POST /flows/{deviceId}?appId={app} | POST /wm/staticentrypusher/json |
| Listar reglas | GET /flows/{deviceId} | GET /wm/staticentrypusher/list/{dpid}/json |
| Borrar reglas | DELETE /flows/{deviceId}/{flowId} · DELETE /flows/application/{appId} | DELETE /wm/staticentrypusher/json · GET /wm/staticentrypusher/clear/{dpid}/json |
| Conectividad declarativa (intents) | POST /intents · GET /intents · DELETE /intents/{appId}/{key} | No existe |
| Aplicaciones (módulos) | GET /applications · POST /applications/{name}/active · POST /applications?activate=true (.oar) | GET /wm/core/module/loaded/json |
| Configuración de una app | GET/POST /configuration/{componente} | No existe |
| Estadísticas | GET /statistics/ports · GET /statistics/flows/link | GET /wm/core/switch/{dpid}/{stat}/json |

**Ejemplo de static entry con la API REST:** la regla DROP que instala rbac_app.py (función deny() del Anexo A) es un POST /flows/{deviceId} con un selector (match) y un treatment (acciones) vacío.

**Dónde consultar la documentación de las APIs:**

- **Documentación oficial de ONOS y µONOS:** https://docs.onosproject.org. La antigua wiki de ONOS (wiki.onosproject.org) ya no está disponible; este sitio de la Open Networking Foundation es el punto de partida para ONOS classic (sección Developers) y para los subsistemas de µONOS, como onos-config (gNMI/OpenConfig).
- **API REST de su controlador (ONOS 2.7):** http://localhost:8181/onos/v1/docs/ con el túnel SSH. Lista todos los recursos con sus parámetros y permite probarlos.
- **Guía de la API REST de ONOS classic:**

https://andrea-campanella.github.io/onos-classic-wiki/onos/guides/developer-guide/appendix-b-rest-api/ (copia comunitaria de la antigua wiki).

- **Código fuente de ONOS:** https://github.com/opennetworkinglab/onos (repositorio oficial, archivado).
- **Floodlight (laboratorio 4):** https://floodlight.atlassian.net/wiki/spaces/floodlightcontroller/pages/1343539/Floodlight+REST+API